ERGALICS STUDIO · 技术文档

04 四大工作模式

让每一次科学计算，都发生在离思想最近的地方

Ergalics Studio 的工作台提供四种模式，由顶栏的 `[标准 | 流程 | 积木 | 代码]` 分段开关切换。四种模式共享同一套数据语义（DataTable）、同一套插件渲染器，后三种模式更共享同一份中间表示（IR）。切换模式时项目数据、插件状态与参数面板原样保留，上下文零损耗。

官网
snowleopard-io.github.io/ErgalicsStudio

GitHub
github.com/SnowLeopard-io/ErgalicsStudio

Gitee
gitee.com/cnt-code/ergalics-studio

本篇章节

一 标准模式

二 流程模式

三 积木模式

四 代码模式

五 三模式互转

目 录

一、标准模式 **03**

二、流程模式（Flow） **04**

2.1 区块目录（38 个内置区块） 04

2.2 编译器与执行器 05

2.3 结果预览 06

2.4 示例管线 06

三、积木模式（Blockly） **07**

四、代码模式（Monaco • Python / R / JavaScript） **09**

五、三模式互转（共享 IR） **11**

Ergalics Studio 的工作台提供四种模式，由顶栏的 `[标准 | 流程 | 积木 | 代码]` 分段开关切换。四种模式共享同一套数据语义（DataTable）、同一套插件渲染器，后三种模式更共享同一份中间表示（IR）。切换模式时项目数据、插件状态与参数面板原样保留，上下文零损耗。

工作台之外还有第二条主线：**科研工具集**（顶栏"分析"按钮与"科研"启动器，或欢迎页启动网格进入）。它由 26 个任务式工具组成，与工作台共享同一个项目与同一组内核，但不使用模式切换——默认以统一 ToolShell 外壳嵌入工作台中央区打开，也可经 `/studio/:toolId` 独立页访问；六组职责见 05 篇。

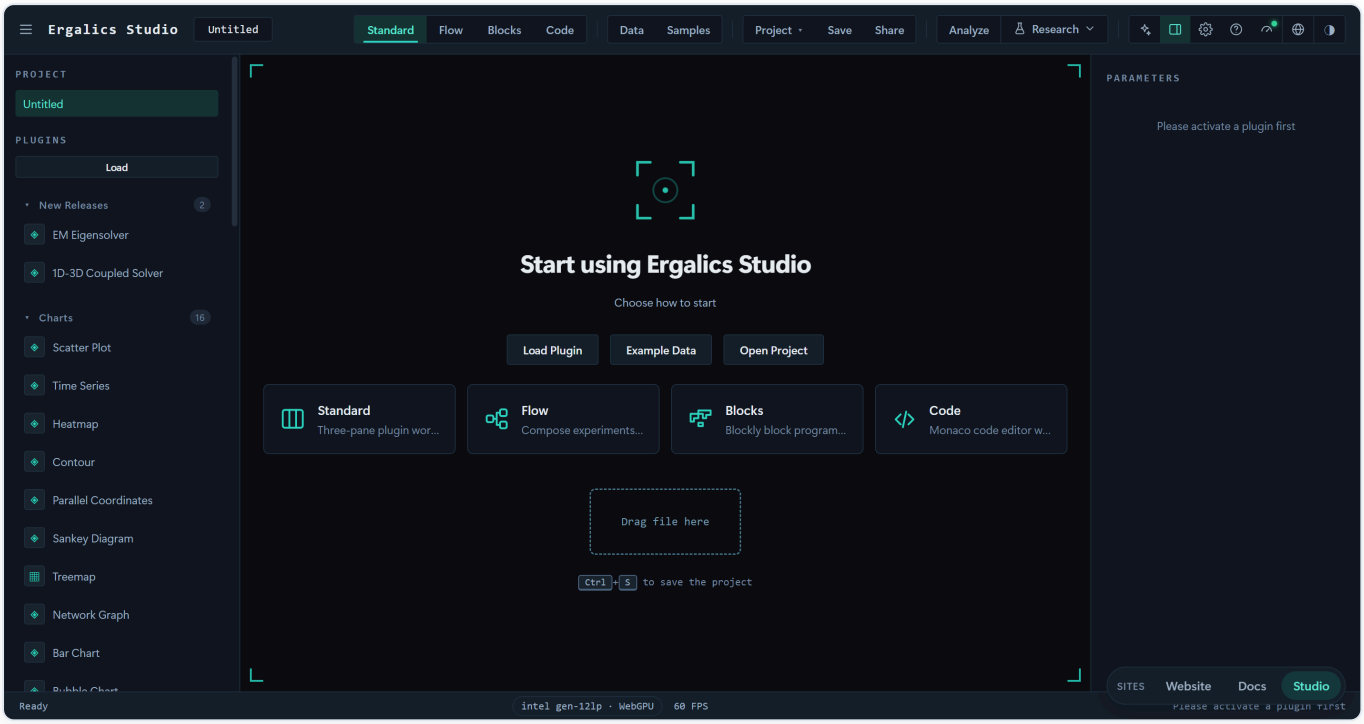
模式	面向人群	核心体验
标准 Standard	已经知道该用哪个插件的人	拖入数据文件，自动识别格式并路由到匹配插件，即刻看到可视化
流程 Flow	管线搭建者	用 38 个内置区块搭出可视化数据流管线，拓扑排序执行，逐节点查看输出
积木 Block	学习者与想要命令式体验的人	类 Scratch 积木编辑器，唯一入口是绿色"运行时"帽子区块
代码 Code	真实脚本用户	Monaco 编辑器，Python 走 Pyodide 中真实的 CPython，R 走 vendored webR 0.6.0 运行时（资源缺失时回退 IR 引擎），JavaScript 在进程内 IR 引擎执行

一 标准模式

默认落地体验。工作台为模块化外壳：顶栏（品牌与项目下拉、四模式切换、数据与科研入口）、左侧活动栏在文件、Git 与插件三个视图间切换（侧栏可调宽、可收起）、中央标签区（激活插件画布、文件预览、按文件分派的编辑器与内嵌科研工具以标签页共存，首次启动为拖放区）、右侧参数面板（把激活插件声明的参数转成响应式表单），状态栏常驻显示 GPU 可用性与性能指标。文件拖入后由宿主按魔数与扩展名路由，多插件匹配时弹选择框。

顶栏动作按语义分簇，以分隔线区隔：项目名下拉（上半区切换最近项目，下半区集中新建 / 打开 / 打开文件夹 / 重命名 / 另存为 / 删除 / 导出日志）、`[标准 | 流程 | 积木 | 代码]` 模式开关（旁有 IR 同步状态点）、`[数据 | 示例]` 数据簇（数据按钮带项目文件数徽标）、`[保存 | 分享]` 项目簇、`[分析 | 科研▾]` 科研簇（科研启动器按学科分组列出 25 个启动网格工具，支持搜索过滤与最近使用，见 05 篇）、右侧全局图标簇（命令面板、快捷键、AI 助手、引导、性能、语言、主题，窄屏折叠进 ⋯ 溢出菜单）。

适合"我已经知道哪个插件能回答我的问题，只想把文件指给它"的场景。



标准模式：模块化外壳工作台，中央标签区承载插件画布、文件预览与编辑器，右侧参数面板实时联动，是默认探索入口。

二 流程模式（Flow）

可视化数据流管线编辑器。左侧调色板、中间画布（节点加边）、右侧参数编辑器、底部结果预览。整个图持久化为工作区文件 `.flow.json`（`{ version, kind: 'ergalics-flow', graph }`，经 `src/core/vfs/` 写入），由文件资源管理器的流程编辑器打开，旧的 `blockGraph` 项目字段仍兼容读取，并共享 `.clproj` 的自动保存与分享管线。

2.1 区块目录（38 个内置区块）

类别	数量	区块
数据源 source	4	示例数据、随机数据生成、网格生成、文件加载
变换 transform	5	列选取、列重命名、新增列、归一化、排序
过滤 filter	3	数值范围过滤、条件值过滤、Top-K 选取

类别	数量	区块
数学 math	6	加、减、乘、除（列对列或列对标量）、平方根、绝对值
统计 stats	14	摘要统计、直方图分箱、单样本 / 独立双样本 / 配对 t 检验、单因素方差分析、Mann-Whitney U 检验、卡方独立性检验、相关分析、Cohen's d 效应量、多重比较校正、Bootstrap、蒙特卡洛传播、MCMC 采样
单位 units	2	量纲换算（units.convert）、量纲一致性检查（units.check）
可视化 viz	4	散点、折线、直方图、二维点云（经渲染桥接走插件出图）
绘图 plot	4	折线、散点、直方图、柱状（出版级绘图引擎渲染）

`viz.*` 与 `plot.*` 的分工是刻意的：`viz.*` 把 `RenderedView` 交给**插件渲染器**，与标准模式看到的是同一个插件；`plot.*` 走 `src/core/plot/` 的**纯 TS 矢量引擎**，产出可直接导出 SVG / PDF 的出版级图。数学与统计区块直接调用 `src/core/stats` 与 `src/core/units` 的纯函数实现（详见 07 篇）；区块元数据（名称、描述）内建中英双语，经 `src/blocks/l10n.ts` 统一解析，新增语言是纯数据改动。

控制流区块（if/else、repeat、parallel）刻意推迟——`BlockInstance` 上的 `region` 接缝已就位，后续可作为扩展嵌入而非重构。

2.2 编译器与执行器

编译器是一个纯函数：结构校验（端口匹配、必需输入、类型兼容）、环检测、Kahn 式拓扑排序，错误以结构化 `diagnostics` 返回，画布据此绘制红色边与内联诊断条，全程不抛异常。

执行器带节点级增量缓存与脏值传播失效：修改单个区块的参数，只有它及其下游会重新执行。运行支持取消与并发去重——重复触发运行不会产生并行执行，取消会真正终止进行中的计算节点。

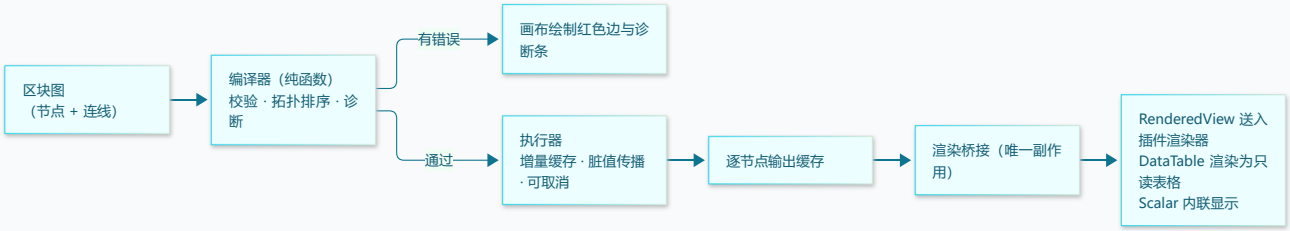
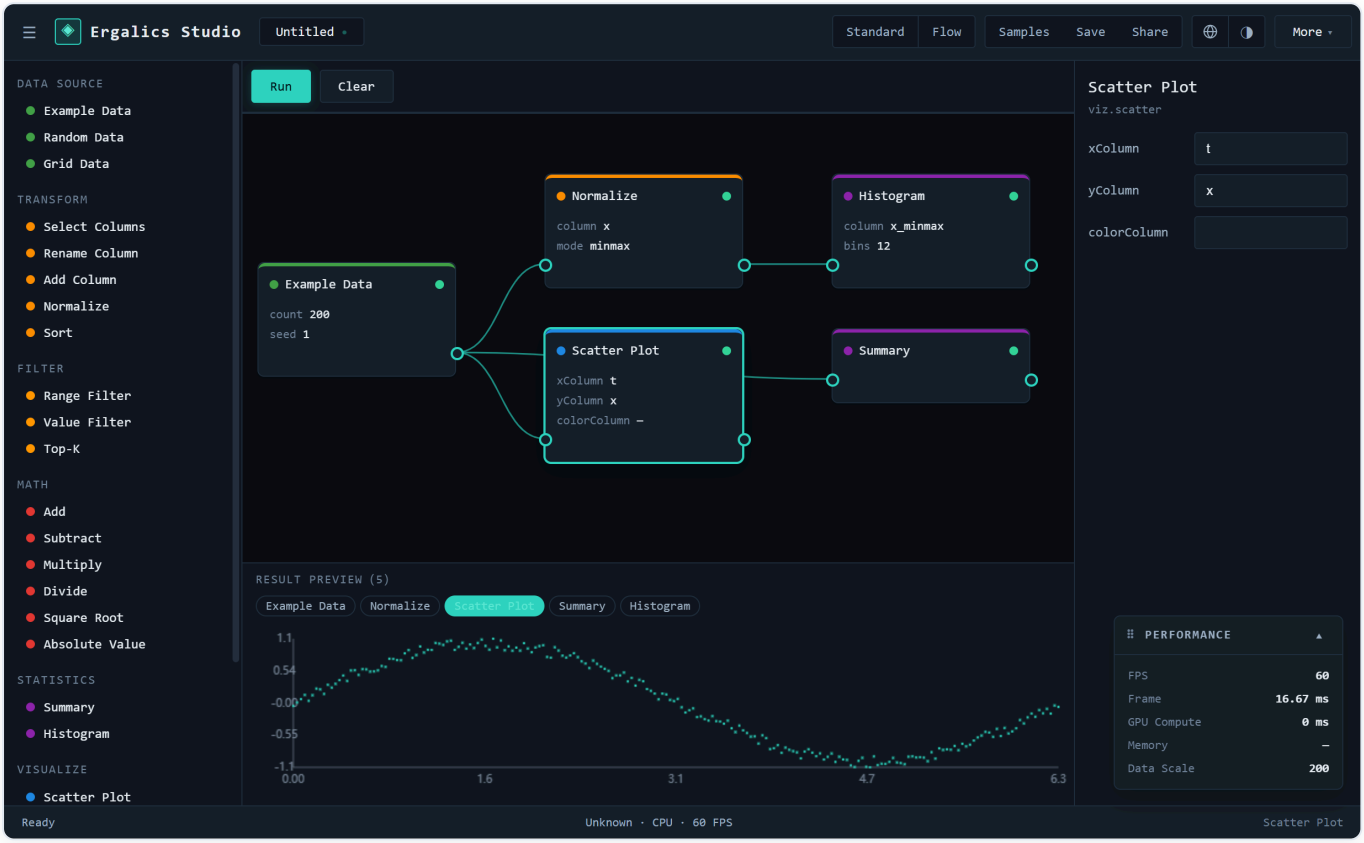


图 1

2.3 结果预览

底部预览随所选节点输出类型自适应：`RenderedView` 走既有插件渲染器（散点、直方图等）；`plot.*` 区块产出的 `svg-plot` 矢量视图以独立 SVG 就地渲染，可直接导出 SVG / PDF；`DataTable` 输出（如 `stats.summary`、`stats.histogram` 的分箱结果）渲染为只读表格，使非可视化输出也看得见；`Scalar` 内联显示。管线有多个输出时由芯片切换器选择检视节点。



流程模式：以 DAG 管线连接数据源 / 变换 / 统计 / 可视化区块，编译后增量执行、就地预览。

2.4 示例管线

8 个流程示例项目以真实 `.clproj` 文件存放在 `examples/projects`（同目录另有 3 个插件演示项目，见 01 篇），构建期经 `import.meta.glob` 自动发现，新增示例只需放入文件并在元数据表登记：

示例	主题
block-01-signal-analysis	信号分析
block-02-random-distribution	随机分布

示例	主题
block-03-grid-scatter	网格散点
block-04-range-filter	范围过滤
block-05-dual-pipeline	双管线对比
block-06-topk-pipeline	Top-K 选取
block-07-binning-stats	分箱统计
block-08-normalize-grid	归一化网格
crystal-lattice-demo	晶格演示
particles-demo	粒子演示
point-cloud-demo	点云演示

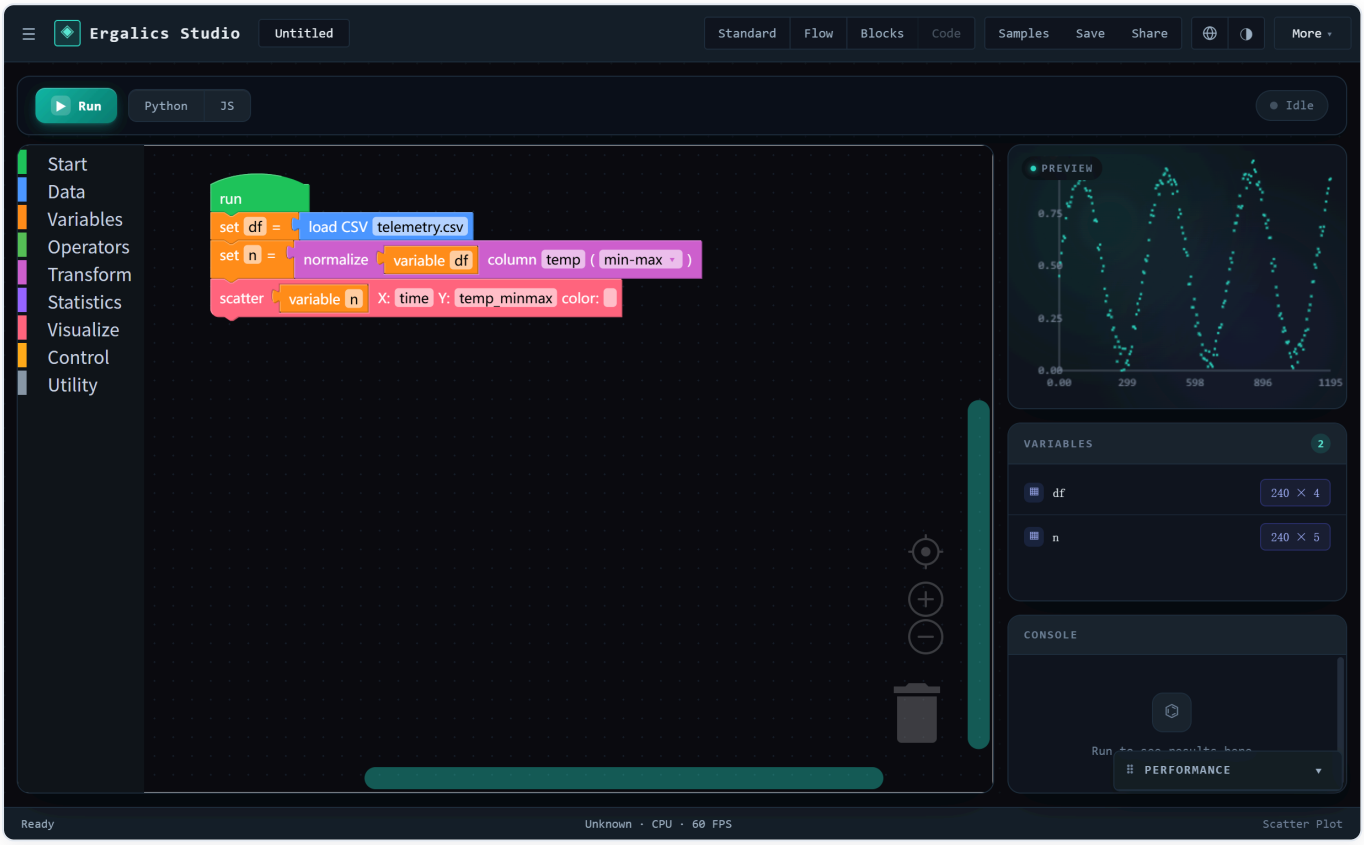
全部 8 个流程示例都通过共享 IR 解释器执行，并由 `examples-roundtrip` 单元测试逐一验证。

三 积木模式（Blockly）

类 Scratch 的积木编辑器，面向学习者与想要命令式体验的用户。唯一执行入口是绿色"运行时"帽子区块，帽子下方未连接的孤立区块永不运行，从根本上杜绝"误执行破损代码"。40 种内置积木按十个类别组织：

类别	积木
开始	运行时帽子区块（唯一入口）
数据	载入 CSV、载入 XYZ、随机数、范围、列表、列表取值
变量	赋值与读取
运算符	数字 / 文本 / 布尔 / 空值字面量、算术、比较、逻辑、一元
变换	归一化、排序、列选取、过滤、添加列
统计	摘要、直方图
绘图	散点、折线、直方图、点云

类别	积木
控制流	如果、重复、当循环、遍历、跳出、继续
函数	定义、返回、调用
工具	打印、原始文本



积木模式：类 Scratch 画布以帽子区块启动脚本，积木拼图编译为共享 IR 执行。

技术要点：

- Blockly 13 驱动画布，包为懒加载分包（约 828 KB），不影响标准与流程模式首屏；媒体资源同源 vendoring，不依赖 CDN。
- 区块名称、提示、下拉选项与工具箱类别经 Blockly 的 `BKY_*` 键系统本地化，切换语言时以重新标注的积木重建工作区，有专门单元测试（`tests/editor/block-i18n.test.ts`）验证。
- 运行按钮提供实时结果预览、变量面板与控制台面板三张卡片。
- 5 个内置示例程序（星系散点图、温度折线图、随机数直方图、标准化后散点、循环打招呼）位于 `src/editor/block/samples.ts`，经顶栏"示例"对话框加载。
- 积木图编译为共享 IR 后由内置解释器执行，解释器调用与代码模式相同的 Studio API。

四 代码模式（Monaco • Python / R / JavaScript）

真正的脚本编辑器。工具栏提供 **Python / R / JS** 分段语言开关与一个引擎徽标，明示当前缓冲区将由谁执行。

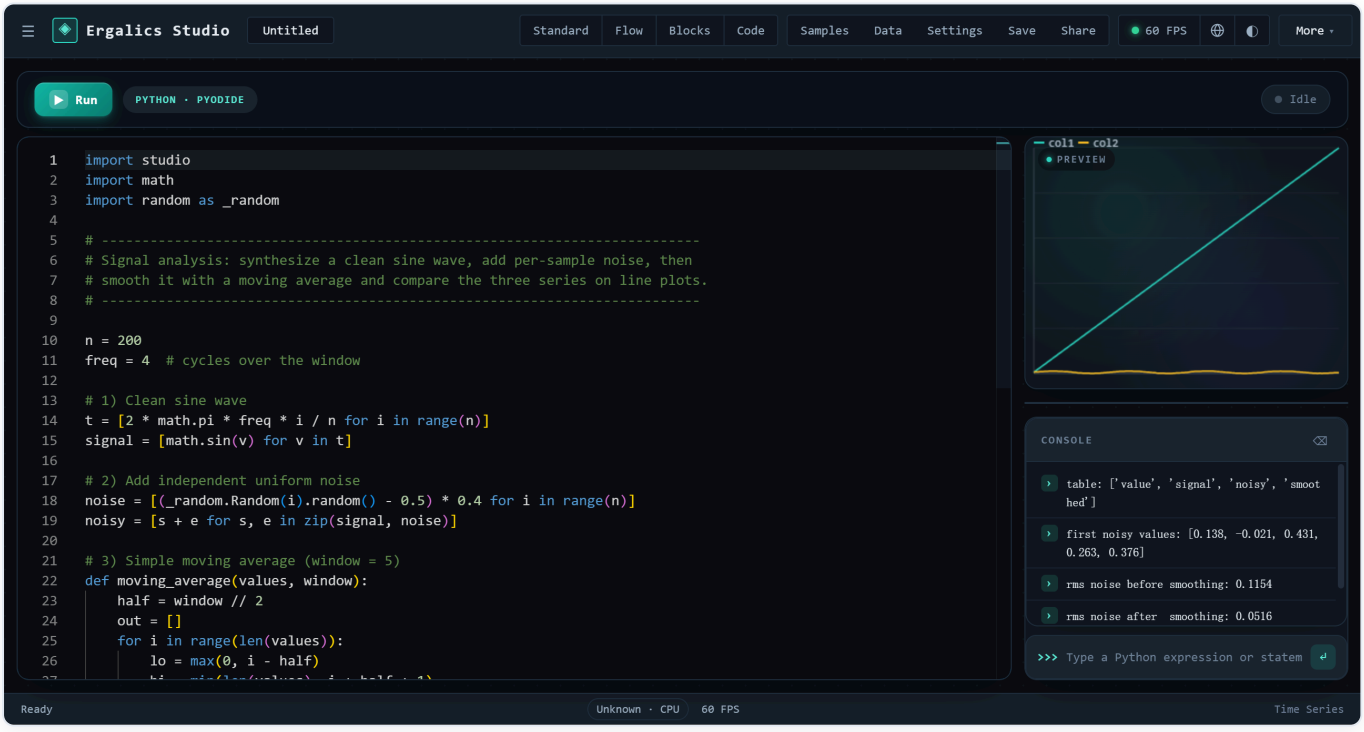
语言	执行引擎	说明
Python	Pyodide Web Worker 中的完整 CPython	自由语法（推导式、f-string、可导入包），`studio` 作为正经可导入模块注入，支持 REPL 单表达式求值
R	vendored webR 0.6.0 运行时（`public/webr/`，资源缺失时回退 IR 引擎）	优先由真实 R 解释器执行；webR 资源缺失或 `studio.*` 桥接降级时回退到共享 IR 引擎；`<-` 赋值
JavaScript	进程内 IR 引擎	缓冲区解析为共享 IR，由与积木模式相同的解释器执行；`const/let/var` 声明

公共能力：

能力	行为
studio 接口跨语言一致	`load / random / range / exampleData / grid / normalize / sort / select / addColumn / addConstantColumn / filter / filterRange / topK / renameColumn / summary / histogram / plot / print / notify / getParam / setParam`。`studio.plot(...)` 经渲染桥接落到与流程模式 `viz.*` 区块完全相同的插件
一键语言互译	切换语言标签时弹出"语言切换预览"悬浮对话框，以内置 Monaco 编辑器呈现可编辑的参考翻译：可直接修改译文后点"应用"换入新语言，或点"放弃并清空"从空白开始，或"取消"留在原语言；放弃清空后出现一条"撤销清空"提示条，一键恢复切换前的代码。切换流程带防清空守卫——翻译结果为空时绝不静默清空非空缓冲区，只有显式"放弃并清空"经一次性放行标记真正清空
中断	停止运行会终止并重启 Worker，失控循环不会卡死页面；Worker 的引导地址与引导 Promise 做了记忆化，重启不重复初始化
快捷键	`Ctrl/⌘ + Enter` 运行缓冲区（并停止进行中的 Python 任务）；键盘输入防抖 150 ms 后同步回 IR，使积木与流程在打字过程中保持实时
DSL 边界显式化	R / JS 解析器无法建模的语句保留为原始代码节点，运行时跳过并在控制台报告一次计数；完整语法随时在 Python 标签页可用

方法	说明
load	装载项目数据文件为表格
random / range / exampleData / grid	生成随机数、整数序列、内置示例与网格数据
normalize / sort / select / addColumn / addConstantColumn / filter / filterRange / topK / renameColumn	表格变换
summary / histogram	摘要统计与分箱
plot	绘图（经渲染桥接走插件渲染器）
print / notify	控制台输出与通知
getParam / setParam	项目参数读写

9 个示例程序以真实文件存放在 `examples/code`（EDA 管线、蒙特卡洛估圆、信号分析、遥测探索、星系散点、随机直方图、范围循环、添加列绘图、归一化过滤），经"示例"对话框加载。



代码模式：Monaco 编辑器 + 控制台 / 变量面板，Python 运行于 Pyodide Worker，注入 `studio` 模块。

五 三模式互转（共享 IR）

IR（`src/editor/ir`，含校验、哈希与序列化）是积木、流程、代码三种模式的唯一事实来源，各模式与 IR 之间都有纯函数往返转换：

转换	模块
IR 与流程 DAG 互转	`src/editor/flow/convert.ts`（irToFlow、flowToIR，Kahn 拓扑排序，参数与区块目录 1:1 对齐）
Blockly JSON 与 IR 互转	`src/editor/block/convert.ts`
代码缓冲区解析回 IR	`src/editor/code/parse.ts`（无法解析的行保留为原始代码节点）
IR 生成代码	`src/editor/codegen`（Python、R、JS 三个生成器）
IR 直接解释执行	`src/editor/runtime/interpreter.ts`（调用与流程区块相同的 Studio API）

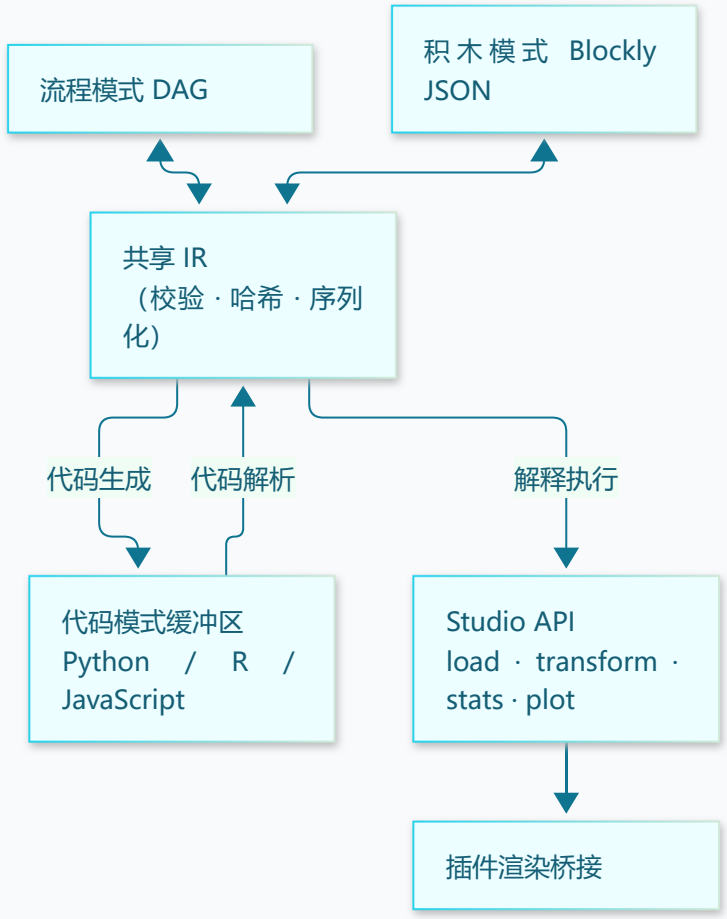


图 2

在流程模式搭一条管线，切到积木就能看到同样的逻辑以积木呈现，跳到代码就能看到生成的 Python；反向亦然。流程编辑会**合并**进 IR 而不是把它拍平（`mergeFlowIR`）：print / loop / if / function 语句留在原位，只替换 DAG 节点；`src/stores/useFlowSync.ts` 中的图签名守卫会忽略注水产生的防抖回声，使反复切换模式不会丢失节点。

代码生成器按目标语言处理方言差异：Python 生成器输出真实 Python 语法，R 生成器处理 R 的语法习惯（循环写法、真值字面量、整除与逻辑运算符等），JS 生成器输出可直接求值的脚本。

往返一致性由 `sync-threeway`、`flow-convert`、`editorStore`、`examples-roundtrip` 与 `roundtrip-all-langs`（Python / R / JavaScript 三语言自由往返）等单元测试钉住，并由 `verify-lang-modes` 端到端套件做真实浏览器的 R/JS 编辑、R→JS 互译、Flow ⇄ Block ⇄ Code 无损循环与真实管线运行验证。

ERGALICS STUDIO · 宗旨

让每一次科学计算，

都发生在离思想最近的地方

Ergalics Studio 把复杂度收进浏览器：安装、配置与数据搬运都不该出现在科研的路上。打开即可计算，数据留在本机，四种工作模式共享同一份数据——一张图、一条流程与一行代码，都是同一次工作的不同表达。

工具越安静，思想就越清楚。我们做的每一处取舍，都为了把注意力还给问题本身。

作者 SnowLeopard-io github.com/SnowLeopard-io

官网 snowleopard-io.github.io/ErgalicsStudio

GitHub github.com/SnowLeopard-io/ErgalicsStudio

Gitee gitee.com/cnt-code/ergalics-studio